Naqsha نقشہ

Technical Overview

The software behind the maps: stack, architecture, engineering strategies, verification and results.

DENSEFUSION · DRONE VIDEO TO SURVEY MAP · OCTOBER 2026

1. What the system does

Naqsha is a local web application and command-line tool built for the OGDCL site survey footage (DJI Mini 5 Pro, 4K at 60 fps). Given a folder of drone videos it works out exactly where the drone flew, which ground each camera frame saw, picks the best frames as photos, stitches them into one top-down map with an elevation model, checks the result and repairs it, without any manual step. There was no recent satellite imagery of the site, so the video itself had to become the map.

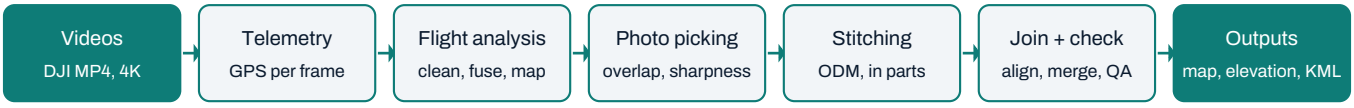


Figure 1. The pipeline. Every stage runs automatically; the web app shows progress in plain language and the same pipeline runs unattended with `./run.sh process <folder>`.

Result on the Shakarparian survey (1 Oct 2026, 7 clips, 21.5 min of video)	
Flight analysis	5.31 km flown, 4 flights from 2 launch sites, farthest point 273 m from takeoff, all inside a 549 m radius circle, 19.7 ha seen with a usable camera angle
Complete map	849 photos picked from 3 usable clips, all 849 fitted into one consistent model, 8.3 ha at 2.5 cm/pixel, plus an elevation model and 3D point cloud
Accuracy	reprojection error 0.99 px; photo positions agree with GPS to 0.4 m (median), 2.4 m (95%); map parts aligned to each other to 0.23 m
Effort	about 6.5 hours on an M2 MacBook (8 GB, CPU only), in 21 memory-sized parts, unattended

2. Technology stack

Layer	Technology	Used for	Where
Video	FFmpeg / ffprobe, VideoToolbox (Mac) or CUDA (NVIDIA) decoding	probing, extracting the telemetry track, quality scan, exact frame export	video.py
Telemetry	own protobuf decoder (no schema needed)	per-frame GPS, height, attitude, gimbal, speed from DJI's embedded 'djmd' track; DJI .SRT sidecars	telemetry/
Geo maths	NumPy, pyproj (UTM), Shapely, Rasterio/GDAL	GPS fusion, camera footprints, coverage, terrain sampling, raster checks	analysis.py, geo.py, coverage.py
Terrain	Copernicus GLO-30 DEM (public, 30 m)	true height above ground, altitude repair	terrain.py
Image checks	OpenCV (SIFT, FLANN, MAGSAC, homography)	blur score, overlap link check, repetitive-texture guard	selection.py
Geotagging	piexif + hand-written XMP	GPS, camera angles, speed, accuracy written into each JPEG	geotag.py
Photogrammetry	OpenDroneMap 3.6 (OpenSfM, OpenMVS, mvs-texturing) in Docker	structure-from-motion, dense surface, orthophoto, DSM, point cloud	odm.py
Merging	ODM's merge code, GDAL	blending parts along cutlines, distance-weighted DEM merge	odm_merge_script.py
Backend	Python 3.12, FastAPI, Uvicorn, SQLite (WAL)	REST API, job queue, persistence	api.py, jobs.py, db.py
Map tiles	rio-tiler on Cloud-Optimised GeoTIFFs	live map tiles straight from the results	tiles.py
Frontend	React 19, TypeScript, Vite, Tailwind CSS, MapLibre GL; Archivo and Noto Nastaliq Urdu fonts, self-hosted	the web app, map, legend, progress, help bubbles	frontend/src
Quality	pytest (26 tests), ruff	geometry, fusion, selection, geotags, QA, memory model	tests/

3. Architecture

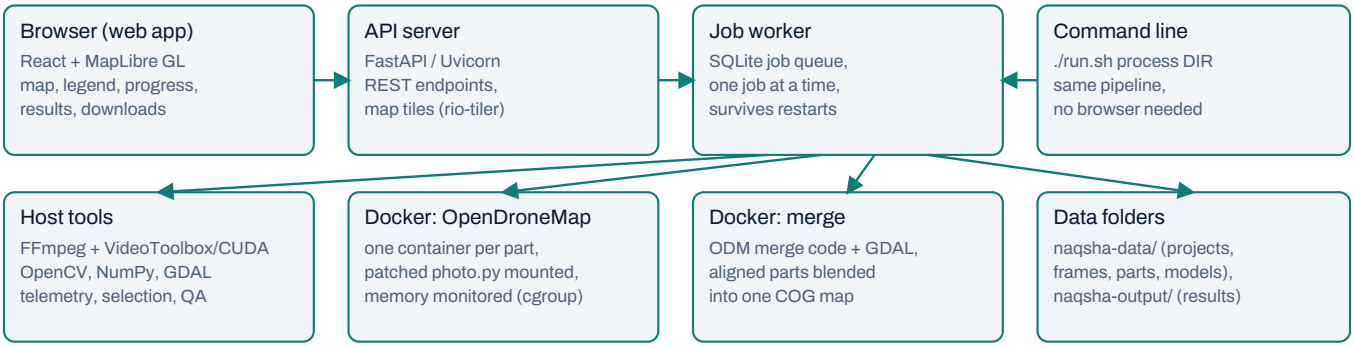


Figure 2. Components. The browser and the command line drive the same job queue; heavy work runs in Docker so the same code runs on a Mac, a Linux GPU server or Windows.

How the stitching itself works. Structure-from-motion (OpenSfM) finds the same points in overlapping photos and solves where every camera was and how it was turned, using the GPS and camera angles as starting values. Multi-view stereo (OpenMVS) then measures depth for millions of pixels, giving a dense 3D surface. The photos are projected straight down onto that surface, which is what turns angled video frames into a true top-down map, and the surface heights become the elevation model. Naqsha's job is everything around this: giving the engine the right photos with exact positions and angles, keeping it inside the memory available, and checking and joining what comes out.

4. Engineering strategies

Reading the GPS hidden in the video. DJI stores a protobuf record for every video frame. With no published schema, the decoder reads raw field numbers. The Mini 5 Pro layout was verified against the GPS track, and for any other drone the layout is inferred from behaviour: coordinate pairs, the velocity whose size follows GPS speed, the angle that follows the GPS course, the height whose change follows vertical speed. Hiding the known layout, inference rediscovers it exactly.

GPS jumps and latency. The GPS track contained persistent steps of up to 21 m. Position is rebuilt as the drone's own velocity integrated over time (centimetre-precise locally) anchored to a 20-second robust median of GPS. Frames near a jump get a wider position uncertainty so the reconstruction trusts the images there.

Altitude repaired. The drone's absolute altitude differed by 44 m between batteries flown from the same spot. Height is rebuilt as the terrain height at takeoff (Copernicus DEM) plus the barometric height, which is consistent to decimetres.

Choosing photos from 60 fps video. Frames are kept by the real ground overlap of their footprints (85%, as forest studies recommend), the sharpest frame in each window, and only while the camera points down, the drone is high enough and is not turning fast (rolling-shutter skew). The camera heading comes from the aircraft yaw because the gimbal yaw carries an arbitrary offset per flight.

Repetitive texture (tree canopy). Every pair of neighbouring photos is feature-matched and the image motion is compared with the GPS/IMU motion. Weak or disagreeing pairs get extra photos. During reconstruction, matching is limited to GPS neighbours, and photos that end up far from their GPS position are flagged and removed by an automatic repair run.

Fixing a bug in OpenDroneMap. ODM 3.6 converts heading to camera angles through Earth-centred axes, which rotates every camera by 90 degrees plus the longitude (163 degrees here). The first map collapsed to 346 points. Naqsha mounts a one-line corrected copy of the faulty file into the container, verifies it by running ODM's own code, and passes exact angles through a geo file: 92,000 points on the same photos. If a future ODM fixes the bug, the probe notices. The fix was submitted upstream as OpenDroneMap pull request #2078.

Fitting in 8 GB of memory. ODM needs about 4 GB for 40 images by its own guidance. Naqsha learns a memory model per quality level (peak = base + slope x photos, fitted over every run), splits the survey into compact overlapping parts that fit, and re-plans when it learns more. A part that still runs out of memory is split and retried; if Docker itself stops, it is restarted. Memory is measured from the container's cgroup (file cache excluded).

Disk and resuming. Each part's scratch data (about 3 GB) is deleted as soon as it finishes, keeping only what joining and checking need. Every finished part carries a marker, so a retry or an automatic repair reuses all unchanged parts.

Joining the parts. Neighbouring parts share photos. Their positions in each part give one offset per part, solved by robust least squares with the mean held at zero, then ODM's own cutline blending and DEM merging produce one map. Part files are never edited, so parts can be shared between runs.

Self-checking. After each map: photos fitted, separate pieces, misplaced photos, gaps against the ground actually flown, reprojection and GPS agreement, combined into a grade with a repair plan.

Hardware. Video decoding uses the Mac's VideoToolbox or NVIDIA CUDA. When an NVIDIA GPU is present and Docker can reach it, stitching switches to ODM's GPU build automatically. Apple's GPU is not reachable from Docker, so on a Mac stitching runs on the CPU.

5. How each claim was verified

Claim	Check
Each exported photo is exactly the frame its GPS belongs to	frame exported by Naqsha vs independent decode of the same frame number: pixel difference 0.0
Heading, camera model and height are right	image motion between photo pairs vs motion predicted from telemetry: agree to 1.07x scale, a few pixels error
Telemetry fields mean what we think	aircraft yaw matches the GPS course within 1 degree; velocity integrates to GPS displacement (ratio 1.00)
Orientation fix is correct	ODM's own code, run inside the container with the patch, reproduces the expected angles; 2,000 random cases within 0.000001 degrees
Layout inference works for other drones	with the known layout hidden, inference finds the same fields and scales on real clips
Joining parts does not damage inputs	checksums of part files identical before and after merging
Chunked map is as good as one run	same 296 photos: 4 joined parts filled 91% of the area vs 86% in one run, GPS agreement 0.35 m vs 0.9 m
Code keeps working	26 automated tests (geometry, GPS fusion, selection spacing, geotags, QA maths, memory model, chunk plans)

6. Files the system produces

File	Format	Contents
map.tif	Cloud-Optimised GeoTIFF, UTM 43N	the stitched top-down map, 2.5 cm per pixel, with transparency outside the mapped area
elevation.tif	GeoTIFF (float)	surface height for every pixel (ground, trees, buildings), metres above sea level
flight.kml / track.geojson	KML / GeoJSON	flight paths per clip, takeoff points, for Google Earth or GIS
coverage.geojson	GeoJSON	ground each camera saw: straight-down, usable, everything
quality.json	JSON	the full quality report: photos fitted, errors, gaps, part alignment
point cloud (.laz)	LAZ	the 3D points, downloadable from the web app

7. The result

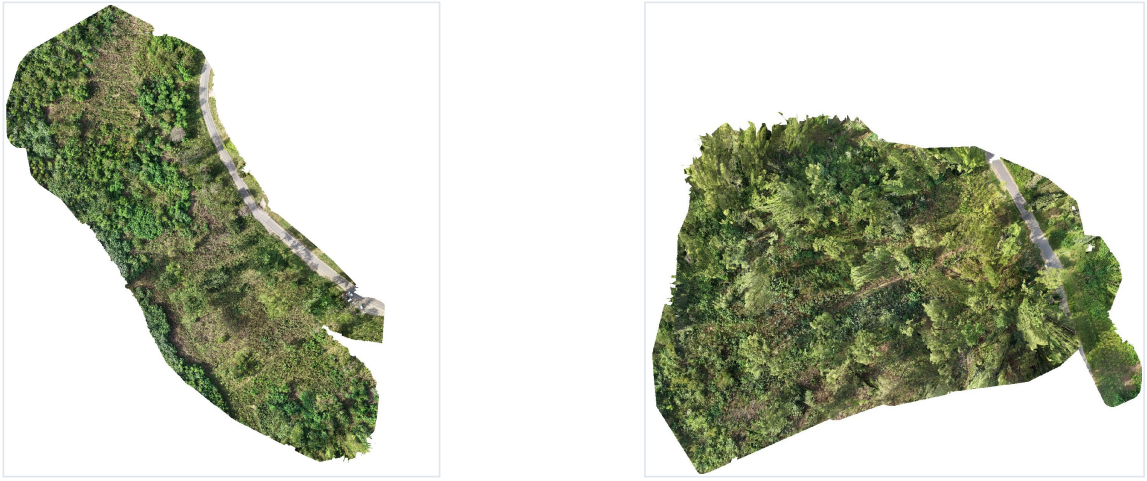


Figure 3. The two mapped areas from the complete map (downscaled 9x). Left: straight-down clip 0033. Right: angled clips 0035 and 0036, usable because of the orientation fix.

Tree counting

Canopy height is the DSM minus a bare-ground surface from SMRF (Pingel et al. 2013) on the DSM's minimum surface. Crowns come from DeepForest 2.1 (RetinaNet, Weinstein et al. 2019): pretrained at 10 cm/px, and at 5 cm/px once taught on the site, because a 1 m sapling is about 10 px at 10 cm, below the detector's smallest anchor. Three methods (image, image checked against height, height only) are scored by F1 against hand-marked squares; random test squares only grade, teaching squares only teach. Active learning asks for 12 m patches where crown confidence sits at the cut-off or image and height disagree; whole patches are marked so no unlabelled tree is taught as background. A height filter decides what is reported. On two held-out patches of young trees (61 trees), F1 rose from 0.23 (pretrained, 10 cm) to 0.49 (pretrained, 5 cm) to 0.62 (taught on 124 trees, 5 cm). Teaching is cumulative and per site: every run retrains from the site's starting point on all squares and answers marked so far, so random squares and active learning feed one model; a new project starts from plain DeepForest unless told to start from another site's model. Shakarparian, model v4 (213 marked trees): 3,638 woody plants from 1 m; 1,442 trees at least 4 m tall; hand-graded accuracy pending.

8. Project layout and running it

`Stiching/naqsha/`: backend/naqsha (Python package, about 5,000 lines: telemetry, analysis, selection, geotag, odm, odm_compat, memory, chunking, qa, pipeline, canopy, trees, api, cli), frontend/src (React app, about 1,500 lines), run.sh, README.md. Data lives in `Stiching/naqsha-data/`; finished maps are also copied next to the videos in `naqsha-output/`.

```
./run.sh                              # web app on http://127.0.0.1:8765
./run.sh process "<video folder>"     # videos in, finished map out, no clicks
cd backend && .venv/bin/python -m pytest -q tests
```

9. Limits and next steps

- Absolute position is consumer-GPS grade (typically 2 to 5 m). Survey-grade accuracy needs ground control points; ODM supports them and a point-picking screen is the natural next feature.

- Dense canopy and moving foliage leave gaps (12% of the flown area here). Higher overlap (90%+) or higher flights reduce them.
- CPU-only stitching on a laptop is slow (hours). The GPU machine path exists and is detected automatically.
- Possible upgrades from the literature: learned matchers (SIFT + LightGlue) for low-texture scenes, global SfM (GLOMAP) for speed, a GCP editor, and a manual seam editor.

10. Development log: problems met and how they were solved

Problem observed	Root cause	Fix
Image motion 3x larger than telemetry predicted	a clip without its own takeoff mixed GPS altitude with DEM ground (80 m instead of 35 m)	fallback ground taken from the DEM at the lowest flown point
Picked frames 0.05 s apart	after a turn, stale candidates from before the turn were chosen	window cleared after every pick; turns choose only recent frames
GPS steps of 21 m	GNSS re-convergence inside the flight controller	velocity dead-reckoning anchored to a robust 20 s GPS median
Absolute altitude off by 44 m	GPS vertical error captured at each takeoff	DEM ground at takeoff + barometric height
ffmpeg failed on 248 frames	expression nesting limit in ffmpeg's select filter (~100 terms)	frames exported in chunks of 50, count verified
First map: 346 points, 78% gaps	ODM's yaw-to-angle conversion bug (heading off by 163 degrees)	verified one-line patch mounted into the container
ODM split-merge crashed	track-building bug in ODM's own submodel mode	own chunking with independent ODM runs, merged with ODM's code
Standard quality ran out of memory	texturing peak above the 3.8 GB Docker had	learned memory model, chunking, split-and-retry
Docker Desktop crashed at 6 GB	macOS starved of memory on an 8 GB machine	budget capped at host RAM minus 3 GB; Docker restarted automatically
Memory looked 4 GB while reading photos	docker stats counts file cache	cgroup anonymous memory measured instead
Disk would have filled (17 x 3 GB)	every part kept its scratch data	finished parts pruned to what merging needs
Merge failed on finished parts	ODM writes Cloud-Optimised GeoTIFFs that GDAL will not edit in place	shifted copies merged; originals never touched
Elevation layer crashed the web page	numpy integers in the legend JSON	converted to plain integers, regression test added